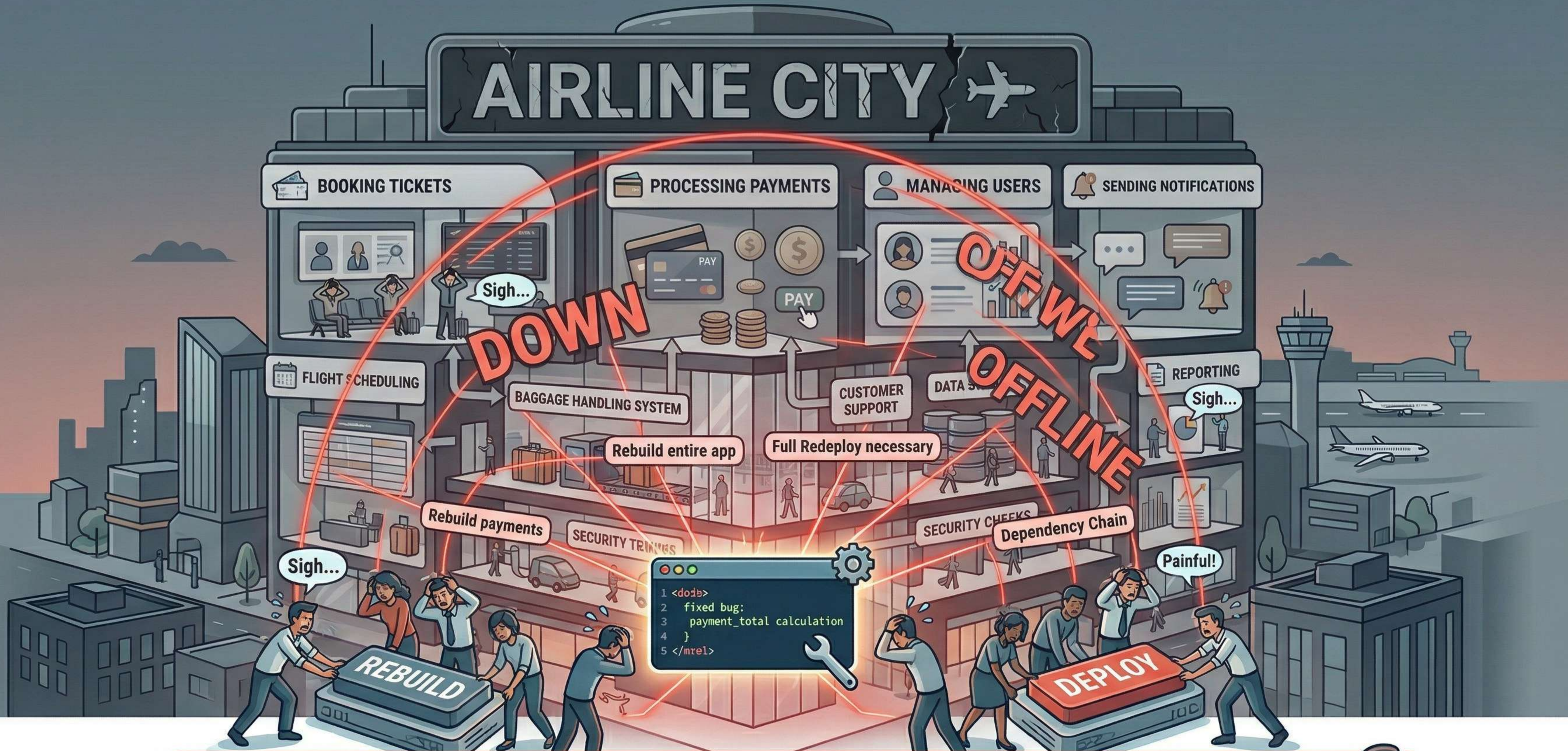


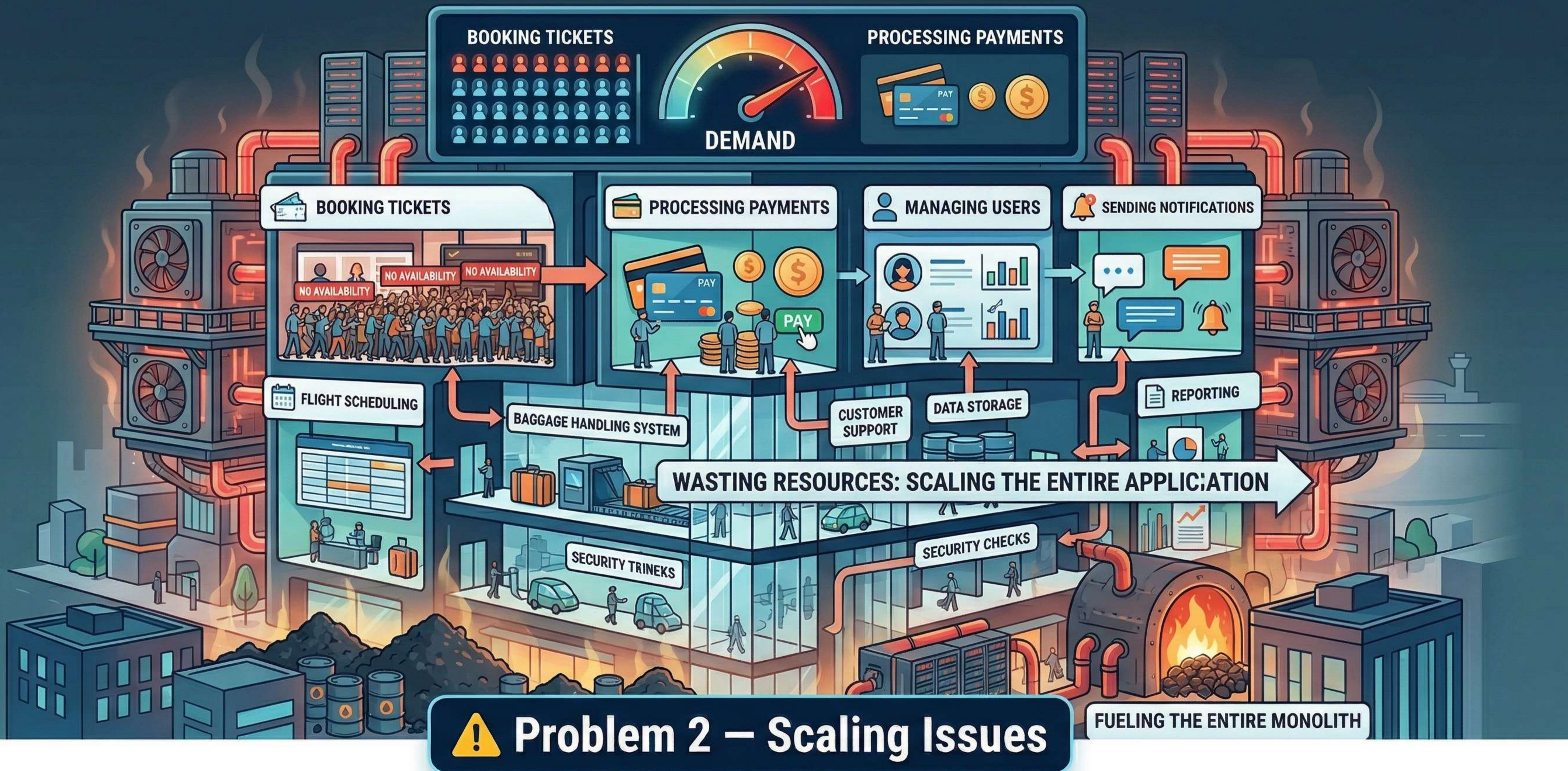


“Mono” means one... one big application that does everything.



PROBLEM 1: SMALL CHANGE = FULL REDEPLOY. Painful! Ouch!

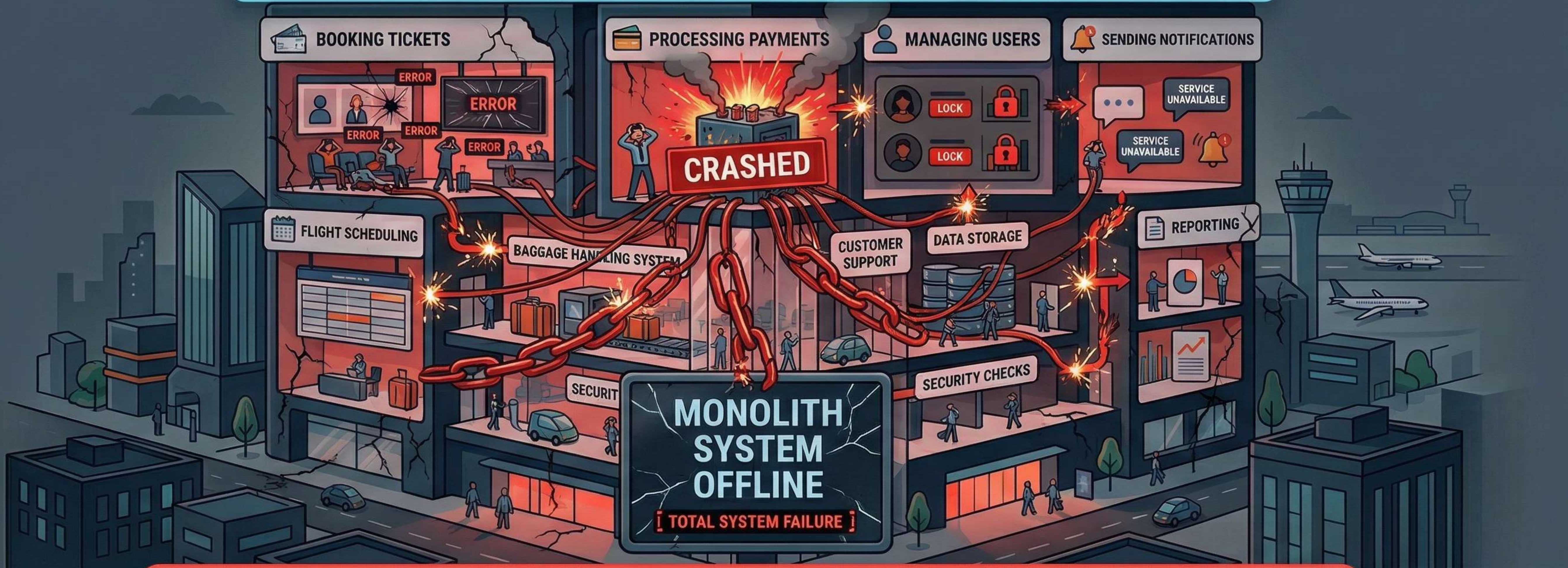




But in a monolith, you can't scale just one part. You are forced to scale the entire application. So even if only booking needs power... you're wasting resources scaling everything.

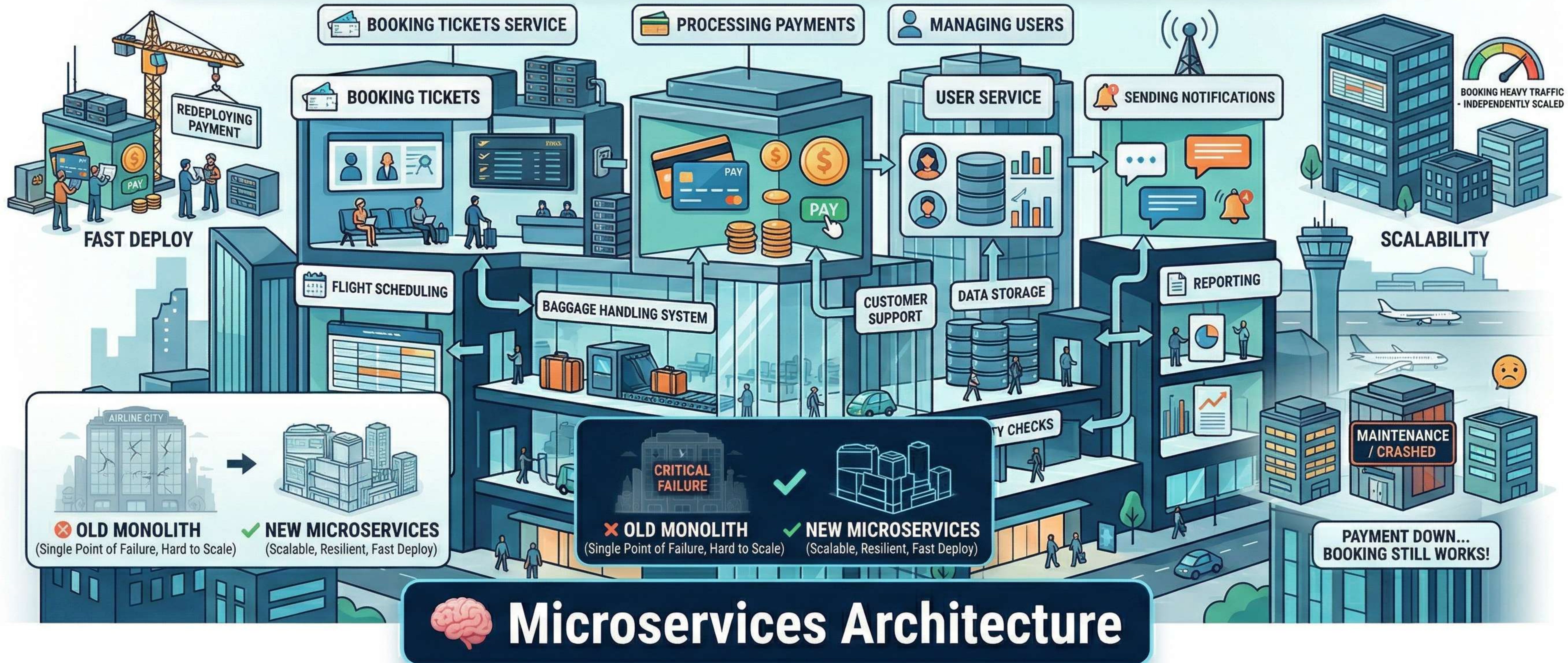
⚠ Problem 3 – Tight Coupling

TIGHT DEPENDENCIES: ONE BREAKDOWN, SYSTEM-WIDE FAILURE



If something breaks in payment, it might affect booking.
If one module crashes, sometimes the whole system goes down 🤖

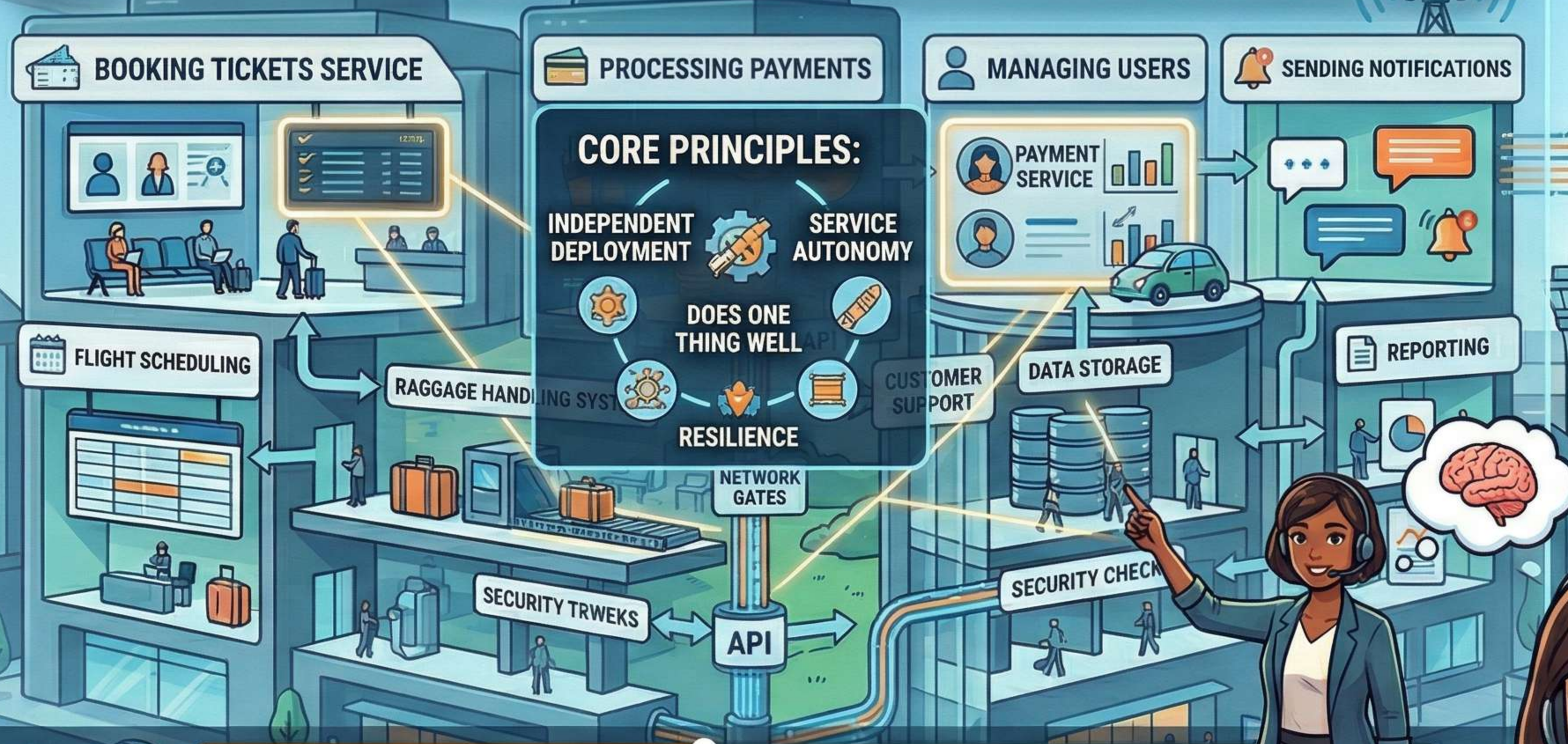
SOLUTION: TRANSITION TO MICROSERVICES ARCHITECTURE



Instead of one big application... multiple small, independent services.

AIRLINE CITY - MICROSERVICES CAMPUS

NEXT CLASS: UNDERSTANDING MICROSERVICE PRINCIPLES



MICROSERVICES 101: NEXT LESSON

👉 What exactly is a **microservice**? And the **core principles** you must follow.
(Because just splitting code into pieces... DOES NOT MEAN you're doing microservices 😊)